copy program, namely 'scp', which stands for 'secure copy'. It uses ssh for data transfer, and uses the same authentication and provides the same security as ssh. Given below is the syntax of the *scp* command:

```
scp [OPTIONS] user@src-host:/src-dir user@dst-host:/target-dir
```

To copy the contents from a remote host to the local one, execute the command given below in a terminal:

```
$ scp -r user@remote-host.com:/remote-dir-path local-dir-path
```

In the above example:
-- *r* option stands for recursive. It will be useful while copying directories
-- *user* is the user name of the remote host
-- *remote-host.com* is the IP address/DNS of the remote host

- *ssh*
  Sometimes, we need to execute a command on the remote host. Obviously, we can log in to that server and execute the command there, but what if we want to capture the output of that command and use it on the local machine? In such a scenario, we can instruct SSH to execute the command on the remote host using the syntax given below:

```
$ ssh user@remote-host.com [COMMAND]
```

For instance, the command given below executes the *ls* command on the remote host:

```
$ ssh user@remote-host.com ls
```

- *rsync*
  rsync is a remote as well as local file-copying tool. The *rsync* utility is used to synchronise the files and directories from one location to another in an effective way.
  To synchronise directories on the local host, execute the *rsync* command as follows:

```
$ rsync -zvr src-dir target-dir
```

In the above example:
-- z option stands for 'enable compression'
-- *v* option stands for 'verbose mode'
-- *r* option stands for 'recursive mode'

To synchronise the remote directory, we have to provide an IP address and user name for that host. For instance, the following command synchronises the local directory with the remote host:

```
$ rsync -zvr src-dir user@remote-host.com:target-dir
```

9) *cron*

We perform many kinds of tasks on a day-to-day basis; for instance, taking backup of important data, checking for updates, and many more. Wouldn't it be great if we automate these tasks? We can achieve this using cron. We can write cron jobs, which will be scheduled periodically. This section provides practical examples of cron.

To list all available cron jobs, execute the command given below:

```
$ crontab -l
```

If any cron job is configured, then it'll be listed here; otherwise, the output will be empty.

Cron jobs are stored in plain text files. To edit those files, we have to use the *crontab* command. But before that, let us understand the cron job format.

A cron job consists of the following six entries:

```
M H  DOM MON DOW  COMMAND
```

In the above example:
-- M stands for 'minutes'
-- H stands for 'hour'
-- DOM stands for 'day of the month'
-- MON stands for 'month'
-- DOW stands for 'day of the week'
-- COMMAND field indicates the command/script to be executed periodically

For instance, to run a job at 5.00 am every week, we can add the following entry:

```
0 5 * * 1 script.sh
```

To add the above entry into cron, perform the following steps:

-- Enter the *crontab -e* command in the terminal and follow the on-screen instructions:

```
$ crontab -e
```

-- Add the cron job entry and save the file.

```
0 5 * * 1 script.sh
```

That's it; and cron will schedule this job at the right time.

10) System monitoring

GNU/Linux provides many utilities to monitor the system. We can monitor memory usage, disk usage, CPU usage and so on. This section discusses some of the popular utilities that can be used to monitor memory and disk usage.